<u>CAD for VLSI Design</u>

指導教授: 陳聿廣

PA1

Benchmark Translator

(結報)

系級 電機四 B

學號 110501544

姓名 翁瑄妤

1. Compilation and Execution

主要在工作站上執行程式，其系統為 Linux 3.10.0、Host 為 140.115.71.87。

執行指令分為以下步驟:

(1) 打開所屬資料夾 (所需資料皆在 PA1 的資料夾裡)

```
cd PA1/
```

(2) 將作業 cpp 檔輸出成執行檔.o 檔

```
g++ -std=c++11 110501544_PA1.cpp -o PA1
```

(3) 執行輸出檔，使 netlist 檔轉換輸出格式至 v 檔 (以 c17 為例)

```
./PA1 c17.netlist c17.v
```

(4) 利用 testbench 檔確認 v 檔內容是否正確 (以 c17 為例)

```
source /usr/cad/cadence/CIC/incisiv.cshrc
source /usr/cad/synopsys/CIC/verdi.cshrc
ncverilog +access+r c17_testbench.v c17.v
```

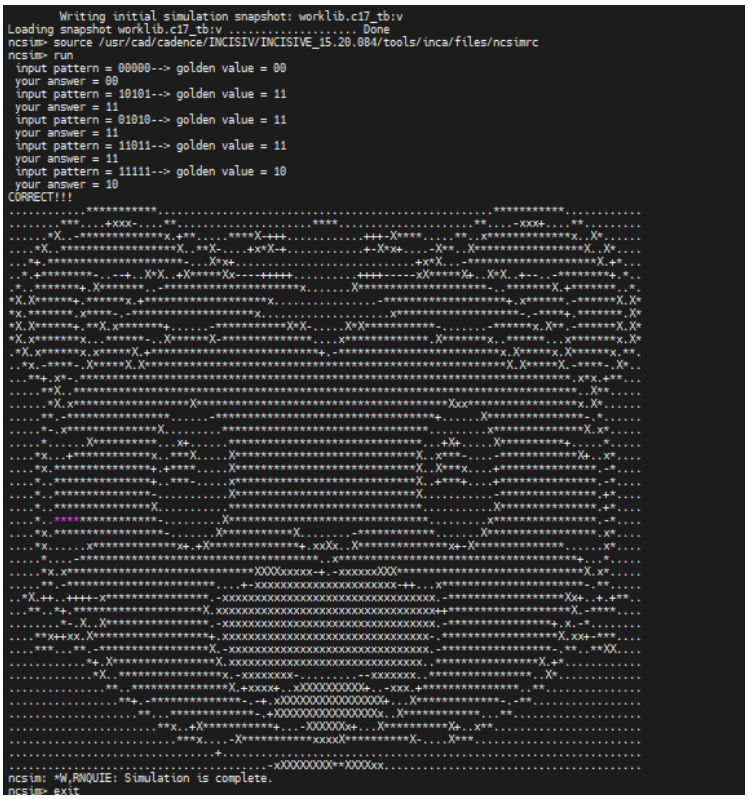
```
Last login: Sun Mar  9 23:32:57 2025 from gateway
[s110501544@ncuee ~]$ cd PA1/
[s110501544@ncuee ~/PA1]$ g++ -std=c++11 110501544_PA1.cpp -o PA1
[s110501544@ncuee ~/PA1]$ ./PA1 c17.netlist c17.v
[s110501544@ncuee ~/PA1]$ source /usr/cad/cadence/CIC/incisiv.cshrc
set INCISIV version: INCISIVE_15.20.084 (default)
[s110501544@ncuee ~/PA1]$ source /usr/cad/synopsys/CIC/verdi.cshrc
set verdi version: 2019.06/ (default)
[s110501544@ncuee ~/PA1]$ ncverilog +access+r c17_testbench.v c17.v
```

2. Completion

以下為執行結果：

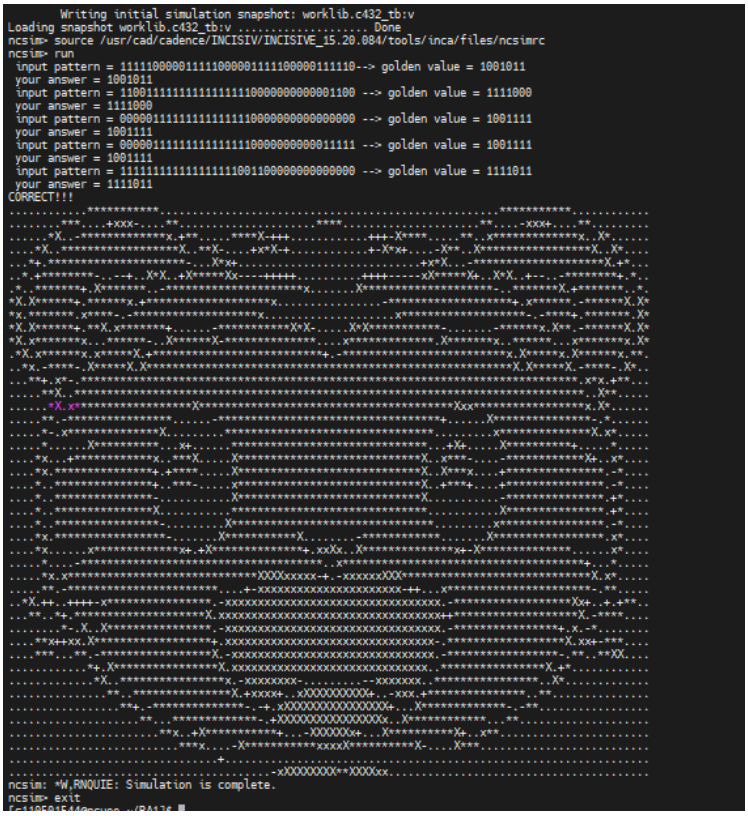
- c17.v

```
        Writing initial simulation snapshot: worklib.c17_tb:v
Loading snapshot worklib.c17_tb:v ..................... Done
ncsim> source /usr/cad/cadence/INCISIV/INCISIVE_15.20.084/tools/inca/files/ncsimrc
ncsim> run
 input pattern = 00000--> golden value = 00
 your answer = 00
 input pattern = 10101--> golden value = 11
 your answer = 11
 input pattern = 01010--> golden value = 11
 your answer = 11
 input pattern = 11011--> golden value = 11
 your answer = 11
 input pattern = 11111--> golden value = 10
 your answer = 10
CORRECT!!!
```



```
ncsim: *W,RNQUIE: Simulation is complete.
ncsim> exit
```

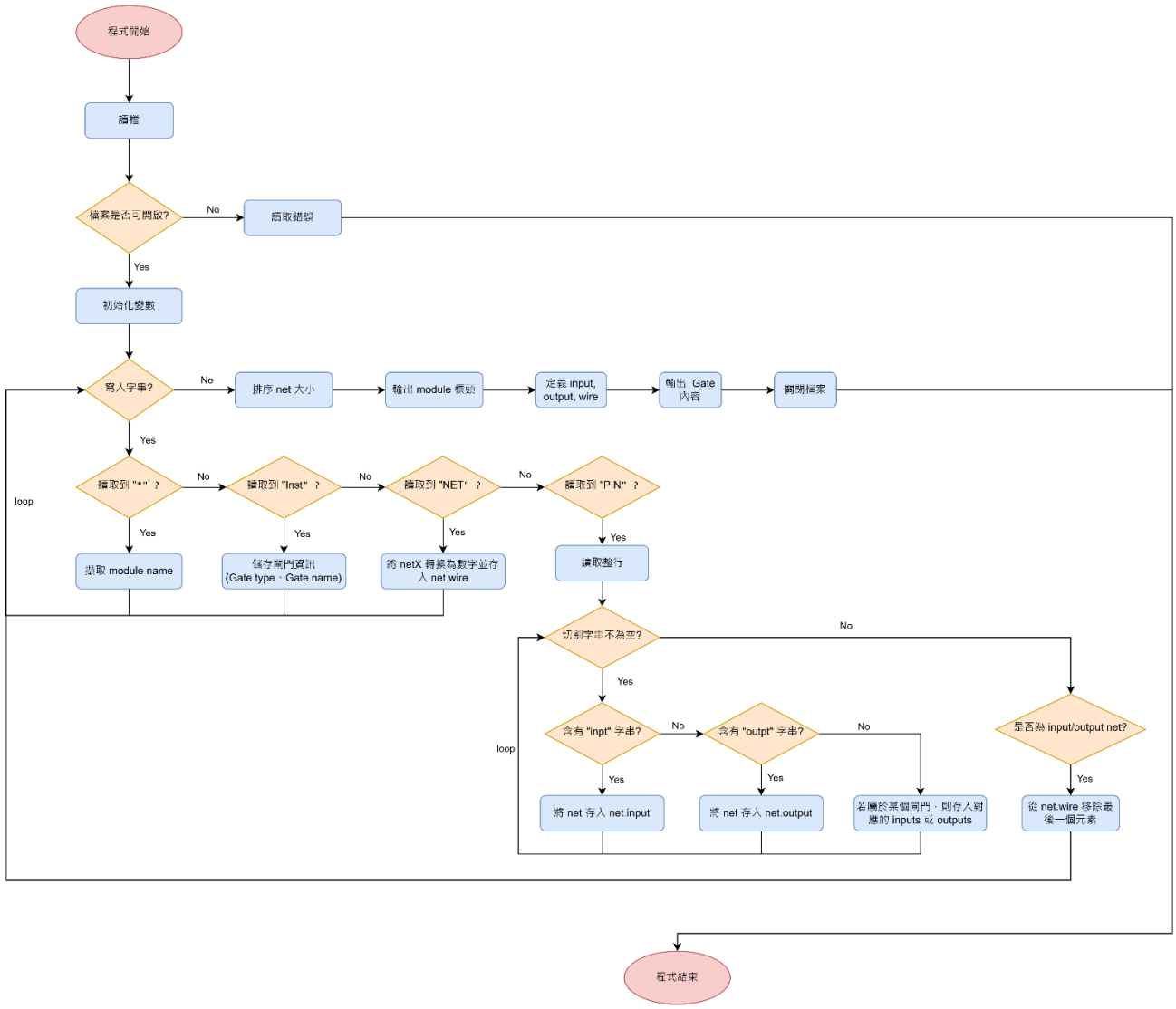
- c432.v



由上面兩張照片可得知，執行出來的兩份 v 檔皆為正確，且指令皆以比照投影片要求執行，因此這份檔案已完成作業中的所有要求。

3. Explanation

(1) 流程圖



(2) 資料結構

在我的程式中，主要利用兩個 struct 結構進行存儲電路資訊。

a. Gate

```
typedef struct GATE{
    string type;
    string name;
    vector<int> inputs;
    vector<int> outputs;
}Gate;
```

每個 Gate struct 儲存一個邏輯閘的資訊，包括 gate 的類型、名稱、gate 的輸入與輸出信號的編號。

b. Net

```
typedef struct NET{
    vector<int> input, output, wire;
}Net;
```

Net struct 負責存儲整個電路信號的資訊 (皆以編號存儲)：

- input：module 的輸入信號
- output：module 的輸出信號
- wire：module 的內部信號（即 gate 之間的連線）

(3) 函式詳細功能與解釋

a. void parse()

◆ pseudo code

```
FUNCTION parse(argv)
    OPEN input file argv[1]
    IF file cannot be opened THEN
        PRINT "File could not be opened"
        EXIT program
    ENDIF

    DECLARE module_name
    DECLARE map gate (stores gates)
    DECLARE struct net (stores input, output, and wire connections)
    DECLARE variable pin_num

    WHILE reading input file line by line
        IF line == "*"
            READ next line and extract module_name
        ELSE IF line == "Inst"
            READ gate name and type
            STORE gate type and name in map
        ELSE IF line == "NET"
            READ net name and remove prefix "net"
            CONVERT net name to integer and store in net.wire
            READ pin_num
        ELSE IF line == "PIN"
            DECLARE in_outpt = 0  (to track input/output nets)
            READ pin connections

            FOR EACH token in line
                IF token contains "inpt"
                    ADD net.wire.back() to net.input
                    SET in_outpt = 1
                ELSE IF token contains "outpt"
                    ADD net.wire.back() to net.output
                    SET in_outpt = 1
                ELSE
                    EXTRACT gateName and pinName
                    IF pinName contains "IN"
                        CALL classify_gate(gate[gateName].inputs,
                            pinName, net.wire, "N")
                    ELSE IF pinName contains "OUT"
                        CALL classify_gate(gate[gateName].outputs,
                            pinName, net.wire, "T")
            ENDFOR

            IF in_outpt == 1 THEN
                REMOVE last element from net.wire
            ENDIF
        ENDIF
    ENDWHILE

    SORT net.input, net.output, and net.wire

    CALL output_verilog(argv, module_name, gate, net)
END FUNCTION
```

◆ 讀取檔案內容與定義 local 變數

```
void parse(char *argv[]){
    ifstream inputfile(argv[1], ios::in);
    if(!inputfile)
    {
        cerr<<"File could not be opened"<<endl;
        exit(0);
    }
    // 讀檔
    string line, name, module_name;
    map<string, Gate> gate;
    Net net;
    int pin_num = 0;
```

- 使用 ifstream 來開啟輸入檔案 argv[1] (從命令列參數取得)，如果開啟失敗則報錯並終止程式。
- module_name：儲存模組名稱，如 c17、c432。
- map<string, Gate> gate：使用 map 結構來儲存所有邏輯閘，key 是 gate 名稱。
- Net net：用來儲存電路中的信號線。

◆ 讀取每一行並解析不同的指令

```
while(inputfile >> line){
    if(line == "*"){
        // 取得 module name
        getline(inputfile, line);
        module_name = line.substr(line.find("module ") + 7, line.length());
    }
```

- 解析 * 行：利用 substr 讀取 * 行中的 ”module ” 後的模組名稱(如* Netlist of the module c432 中的 c432)，儲存到 module_name，此將在 v 檔中的 module 宣告中使用。

```
        else if(line == "Inst"){
            string type;
            inputfile >> name;
            inputfile >> type;
            gate[name].type = type; // save gate type
            gate[name].name = name;     // save gate name
        }
```

- 解析 Inst 行：讀取 gate 名稱 (name) 和 gate 類型 (type)，並存入 gate 映射表中。

```
        else if(line == "NET"){
            inputfile >> name;
            // 把 "net1" 的 "net" 去掉，只留下後面的數字
            name.erase(0,3);
            // 先把 net 存放在 wire 中
            net.wire.push_back( stoi(name) );
            inputfile >> pin_num;
        }
```

- 解析 NET 行：移除 "net" 前綴，將信號名稱轉換為數字並先存入 net.wire（內部訊號線），以方便後續名稱由小到大的排序，以及後面判斷剛剛存在 net.wire 的 net 為電路信號中的 input、output、或 wire。

```
        else if(line == "PIN"){
            int in_outpt = 0;   // 判斷是否為 input/output wire 還是 wire 決定是否要 wire.pop_back() (把 net 移出 wire)
            getline(inputfile, line);
            char *token = strtok(&line[0], ", ");
            while (token != NULL){
                name = token;
                if(name.find("inpt") != string::npos){  // 查找是否含有 "inpt" 字串
                    // 將 net 存放在 input 中
                    net.input.push_back( net.wire.back() );
                    in_outpt = 1;
                }
                else if(name.find("outpt") != string::npos){    // 查找是否含有 "outpt" 字串
                    // 將 net 存放在 output 中
                    net.output.push_back( net.wire.back() );
                    in_outpt = 1;
                }
                else{
                    // 將 pin 分類存放在 gate 中
                    string gateName = name.substr(0, name.find("/"));
                    string pinName = name.substr(name.find("/") + 1, name.length());
                    if(pinName.find("IN") != string::npos)
                        classify_gate(gate[gateName].inputs, pinName, net.wire, "N");
                    else if(pinName.find("OUT") != string::npos)
                        classify_gate(gate[gateName].outputs, pinName, net.wire, "T");
                }
                token = strtok(NULL, ", ");
            }
            if(in_outpt == 1) net.wire.pop_back();  //若為 input/output wire 則把 ner 移出 wire
        }
```

- 解析 PIN 行：讀取連接的訊號名稱，並根據名稱判斷
  - 若在 PIN 行中有出現 "inpt"，則將剛剛存在 net.wire 的 net 儲存至 net.input。
  - 若在 PIN 行中有出現 "outpt"，則將剛剛存在 net.wire 的 net 儲存至 net.output。
  - 其他則視為內部連線，並加入到相對應 gate 中的 input 端 或 output 端 (如為 gatX/INX，則儲存在 gate[“gatX”].inputs 中；若為 gatX/OUTX，則儲存在 gate[“gatX”].outputs 中)。
  - 最後則判定剛剛存在 net.wire 的 net 是否是電路信號中的 input 或 output，若屬實則將 net 從 net.wire 移除。

◆ 排序訊號並輸出 Verilog

```
    // 排序 net 大小
    sort(net.input.begin(), net.input.end());
    sort(net.output.begin(), net.output.end());
    sort(net.wire.begin(), net.wire.end());
    // 輸出檔案
    output_verilog(argv, module_name, gate, net);
}
```

- 對 input, output, wire 進行由小到大的排序，方便輸出 v 檔的閱讀性。
- 呼叫 output_verilog() 來輸出 v 檔。

b. void classify_gate()

◆ pseudo code

```
FUNCTION classify_gate(pin, pinName, wire, in_or_out)
    EXTRACT pinNum from pinName
    EXPAND pin array to accommodate pinNum if needed
    ASSIGN wire.back() to pin[pinNum - 1]
END FUNCTION
```

◆ 功能

```
// 將 pin 分類存放在 gate 中
void classify_gate(vector<int> &pin, string pinName, vector<int> wire, string in_or_out){
    int pinNum = stoi( pinName.substr(pinName.find(in_or_out) + 1, pinName.length()) );
    while ( pin.size() < pinNum )
        pin.push_back(0);
    pin[pinNum - 1] = wire.back();
}
```

- 負責解析 PIN 名稱，將訊號分配到對應 gate 的 input 端 或 output 端 (如為 gatX/IN4，則將此 net 放在 gate[“gatX”].inputs 中的第 4 個位置；若為 gatX/OUT1，則將此 net 放在 gate[“gatX”].outputs 中的第 1 個位置)，若發現位置不夠則先將位置擴充並設定初始值 0。

c. void output_verilog()

◆ pseudo code

```
FUNCTION output_verilog(argv, module_name, gate, net)
    OPEN output file argv[2]
    IF file cannot be opened THEN
        PRINT "File could not be opened"
        EXIT program
    ENDIF

    PRINT Verilog header and module declaration
    PRINT input, output, and wire declarations

    DECLARE map gate_type (to track instances of each gate type)

    FOR EACH gate in map
        CONVERT gate type to uppercase
        GENERATE instance name using gate type and count
        PRINT gate declaration in Verilog format
    ENDFOR

    PRINT "endmodule"
    CLOSE output file
END FUNCTION

END
```

◆ 開啟輸出檔案

```
// 輸出檔案 (轉換成 verilog 的形式)
void output_verilog(char *argv[] , string module_name, map<string, Gate> gate, Net net){
    ofstream outputfile(argv[2], ios::out);
    if(!outputfile)
    {
        cerr<<"File could not be opened"<<endl;
        exit(0);
    }
    outputfile << "`timescale 1ns/1ps" << endl << "module " << module_name << " (";
```

- 使用 ofstream 來開啟輸出檔案 argv[2]（從命令列參數取得）。
- 若檔案無法開啟，則輸出錯誤訊息並終止程式。
- 輸出 Verilog 檔案標頭，包含先前儲存的 module 名稱。

◆ 輸出 module 的 I/O

```
    // 輸出 module 的 I/O
    for(const auto &i : net.input)
        outputfile << "N" << i << ",";
    for(const auto &o : net.output)
        outputfile << "N" << o << ( &o == &net.output.back() ? ");" : "," );
```

- 輸出所有 net.input 的輸入訊號（以 N<number> 命名）。
- 輸出所有 net.output 的輸出訊號，最後一個訊號後面加上 “);” 作結尾。

Ex. 如果 net.input = {1, 2}，net.output = {3}，則會輸出：

module my_circuit (N1, N2, N3);

◆ 輸出 input, output, wire 宣告

```
    // 輸出 input, output, wire 宣告
    outputfile << endl << endl << "input ";
    for(const auto &i : net.input)
        outputfile << "N" << i << ( &i == &net.input.back() ? ";" : "," );
    outputfile << endl << "output ";
    for(const auto &o : net.output)
        outputfile << "N" << o << ( &o == &net.output.back() ? ";" : "," );
    outputfile << endl << "wire ";
    for(const auto &w : net.wire)
        outputfile << "N" << w << ( &w == &net.wire.back() ? ";" : "," );
```

- 輸出所有 net.input、net.output、net.wire 的電路信號。

◆ 輸出 gate 宣告

```
    // 輸出 gate
    map<string, int> gate_type;  // 紀錄 gate type 的數量
    for(const auto &g : gate){
        string str;
        for (int i=0; i<g.second.type.size(); i++)
            str += toupper(g.second.type[i]);
        str += to_string( g.second.inputs.size() + g.second.outputs.size() - 1 );
        outputfile << endl << g.second.type << " " << str << "_" << (gate_type[str]++) + 1 << " (" ;
        for(const auto &o : g.second.outputs)
            outputfile << "N" << o << ", ";
        for(const auto &i : g.second.inputs){
            outputfile << "N" << i << ( &i == &g.second.inputs.back() ? ");" : ", " );
        }
    }
    outputfile << endl << endl << "endmodule";
    outputfile.close();
}
```

- map<string, int> gate_type 用來計算每種閘門的數量，避免名稱重複。
- 將 g.second.type 轉換成大寫 (如果 g.second.type = "and"，則轉換為 "AND")。
- str 變數會儲存閘門名稱+接腳數 (如"AND2"：2-input AND 、"OR3"：3-input OR)。
- gate_type[str]++ 會累計該類型的閘門數量，確保名稱不重複。
- 輸出"("後會先輸出 gate 中的 output 端，再輸出 input 端。
- 最後輸出 endmodule 並關閉輸出檔案。

d. Main function

◆ 主程式功能

```
int main(int argc, char *argv[]){
    parse(argv);
    return 0;
}
```

- main(argc, argv) 負責接收命令列引數：
  - argv[1] 是輸入檔案名稱 (netlist 檔)。
  - argv[2] 是輸出檔案名稱 (v 檔)。
- main() 呼叫 parse(argv) 來解析輸入檔案並處理數據。

4. Hardness

我覺得最困難的點是在於 gate 和 net 之間的關係是多對多，也就是每個 gate 有多個 input 端和 output 端，以及每條 net 可能連接多個 gate。若使用一般的資料結構(如 vector、linked list 等)會較難管理與搜索。因此我使用 map<string, Gate> 來儲存所有 gate 類型及 gate 連接的 input、ouput 端。使用 map 結構存儲資料是因為 map 是 key-value 的結構，可透過搜尋 key 值找到 value 值，因此我利用 gate 名稱當作我的 key 值，如此便可在後續擷取資料時(如 gat6/IN1)，更方便找到 gate 結構位置並存儲其餘資訊，以及最後輸出所有資訊。

5. Suggestions

這次的作業流程以及工作站的建立與執行步驟都非常詳細，並且透過投影片與影片的輔助，使我能夠清楚理解如何執行程式。這份作業幫助我學會如何使用 C++ 來處理輸入與輸出檔案，同時也讓我重新複習了 C++ 的語法與功能。

然而，我認為題目說明的部分可以更完整，而不僅僅是提供圖片與部分輸入資訊。例如，可以明確說明輸入檔案中 gate 的命名規則（如是否為 "gat+數字"）、gate 類型是否有特定限制、pinName 的命名規範，以及輸出檔案中 gate 的名稱是否有特定要求等。此外，若學生在作業過程中遇到問題，希望可以將學生問到的所有問題與解答統一整理並公布在 eeclass 上，方便大家參考與學習。最後，我也希望能提供更多測試案例（test cases），讓我們能夠更全面地測試與驗證程式的正確性與完整性。